

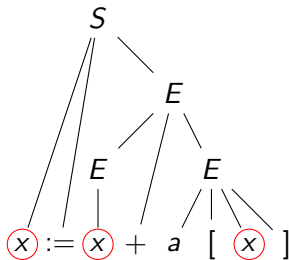
コンパイラ (5)

2026/10/16

記号表

- ▶ 変数宣言：記号定義
- ▶ 参照：記号参照

定義によって得られる情報を記憶しておいて参照の際に利用する
中間コードで変数になっていた部分に数値をわりあてる

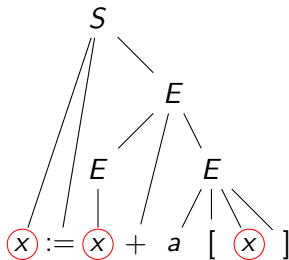


x は構文木上では無関係

記号表

- ▶ 変数宣言：記号定義
- ▶ 参照：記号参照

定義によって得られる情報を記憶しておいて参照の際に利用する
中間コードで変数になっていた部分に数値をわりあてる



x は構文木上では無関係

構文木上の要素の関連
付けて同じアドレスを
使ったコードを生成
⇒ 表に記録

登録情報

- ▶ 名前：識別子
- ▶ 種類：属性、関数名、変数名など
- ▶ 型：記憶領域、引数の数、型
- ▶ 記憶位置：スタック内での相対位置

データ構造

- ▶ 線形リスト 配列： $O(n)$ でアクセス
- ▶ ハッシュ
- ▶ 二分木 $O(\log_2(n))$ でアクセスデータ+左ポインタ (自分より小さいエントリ)+右ポインタ (自分より大きいエントリ)

スコープルールの管理

```
int a=0; int b=5;
void f() {
    int a=2;
    printf("%d\n",a);
}
void g() {
    int a=3;
}
void main() {
    int a=3;
    f();g();
}
```

name	kind	val	lvl	type
------	------	-----	-----	------

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理

```
→ int a=0; int b=5;
   void f() {
       int a=2;
       printf("%d\n",a);
   }
   void g() {
       int a=3;
   }
   void main() {
       int a=3;
       f();g();
   }
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理

```
int a=0; int b=5;  
→ void f() {  
    int a=2;  
    printf("%d\n",a);  
}  
void g() {  
    int a=3;  
}  
void main() {  
    int a=3;  
    f();g();  
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
f	func	v_f	G	void

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理

```
int a=0; int b=5;
void f() {
→   int a=2;
    printf("%d\n",a);
}
void g() {
    int a=3;
}
void main() {
    int a=3;
    f();g();
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
f	func	v_f	G	void
a	var	0	L	int

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理

```
int a=0; int b=5;
void f() {
    int a=2;
→   printf("%d\n",a);
}
void g() {
    int a=3;
}
void main() {
    int a=3;
    f();g();
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
f	func	v_f	G	void
a	var	0	L	int

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理

```
int a=0; int b=5;
void f() {
    int a=2;
    printf("%d\n",a);
→ }
void g() {
    int a=3;
}
void main() {
    int a=3;
    f();g();
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
f	func	v_f	G	void

局所変数 a を破棄

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理

```
int a=0; int b=5;
void f() {
    int a=2;
    printf("%d\n",a);
}
→ void g() {
    int a=3;
}
void main() {
    int a=3;
    f();g();
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
f	func	v_f	G	void
g	func	v_g	G	void

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理

```
int a=0; int b=5;
void f() {
    int a=2;
    printf("%d\n",a);
}
void g() {
→   int a=3;
}
void main() {
    int a=3;
    f();g();
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
f	func	v_f	G	void
g	func	v_g	G	void
a	var	0	L	int

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理

```
int a=0; int b=5;
void f() {
    int a=2;
    printf("%d\n",a);
}
void g() {
    int a=3;
→ }
void main() {
    int a=3;
    f();g();
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
f	func	v_f	G	void
g	func	v_g	G	void

局所変数 a を破棄

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理

```
int a=0; int b=5;
void f() {
    int a=2;
    printf("%d\n",a);
}
void g() {
    int a=3;
}
→ void main() {
    int a=3;
    f();g();
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
f	func	v_f	G	void
g	func	v_g	G	void
main	func	v_0	G	void

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理

```
int a=0; int b=5;
void f() {
    int a=2;
    printf("%d\n",a);
}
void g() {
    int a=3;
}
void main() {
→   int a=3;
    f();g();
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
f	func	v_f	G	void
g	func	v_g	G	void
main	func	v_0	G	void
a	var	0	L	int

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理

```
int a=0; int b=5;
void f() {
    int a=2;
    printf("%d\n",a);
}
void g() {
    int a=3;
}
void main() {
    int a=3;
→   f();g();
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
f	func	v_f	G	void
g	func	v_g	G	void
main	func	v_0	G	void
a	var	0	L	int

v_0, v_f, v_g : 関数の開始命令番地

スコープルール

スコープルール=識別子の有効範囲

C 言語の場合：大域変数、ファイル内大域、関数内局所変数、ブロック内局所変数

Java 言語の場合：public、protected、private、ブロック内局所

スコープルールの実現

演習のコンパイラ：厳密に1パス

- ▶ 局所環境廃棄/復活の方法
- ▶ 上書きのメカニズム

局所変数を完全に上書きしてしまうと復活できなくなる
解決法:

2 以上同じ識別子のエントリを許して参照する順序を決める
局所変数のラベルをつけて局所環境から出たら破棄
(局所関数のレベルが1段階なので非常に単純)

スコープルールの管理: 問題点

```
int a=0; int b=5;
void main() {
    int a=1;
    f();g();
}
void f() {
    int a=2;
    printf("%d\n",a);
}
void g() {
    int a=3;
}
```

name	kind	val	lvl	type
------	------	-----	-----	------

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理: 問題点

```
→ int a=0; int b=5;
   void main() {
       int a=1;
       f();g();
   }
   void f() {
       int a=2;
       printf("%d\n",a);
   }
   void g() {
       int a=3;
   }
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理: 問題点

```
int a=0; int b=5;  
→ void main() {  
    int a=1;  
    f();g();  
}  
void f() {  
    int a=2;  
    printf("%d\n",a);  
}  
void g() {  
    int a=3;  
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
main	func	v_0	G	void

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理: 問題点

```
int a=0; int b=5;
void main() {
→   int a=1;
    f();g();
}
void f() {
    int a=2;
    printf("%d\n",a);
}
void g() {
    int a=3;
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
main	func	v_0	G	void
a	var	0	L	int

v_0, v_f, v_g : 関数の開始命令番地

スコープルールの管理: 問題点

```
int a=0; int b=5;
void main() {
    int a=1;
→   f();g();
}
void f() {
    int a=2;
    printf("%d\n",a);
}
void g() {
    int a=3;
}
```

name	kind	val	lvl	type
a	var	0	G	int
b	var	1	G	int
main	func	v_0	G	void
a	var	0	L	int

エラー f,g が参照できない。

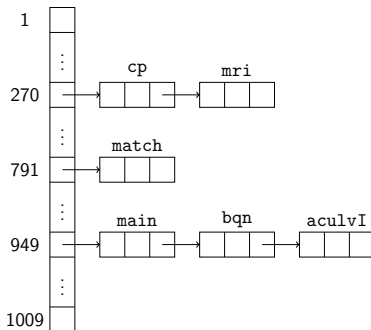
参照の前に宣言されている必要がある

v_0, v_f, v_g : 関数の開始命令番地

ハッシング (Hashing)

中間コード生成: `lod 0,0,x` の `x` のアドレスを記号表を参照する
必要あり: 高速にデータアクセスするテクニック

Open Hashing: $f(x) = f(y)$ となる x, y をリストでつなぐ。



ハッシュ関数 $h(s):s$ に対して数をわりあてる。できるだけばらになるようにする=ハッシュ表につながるリストを短かくする

ハッシュ関数

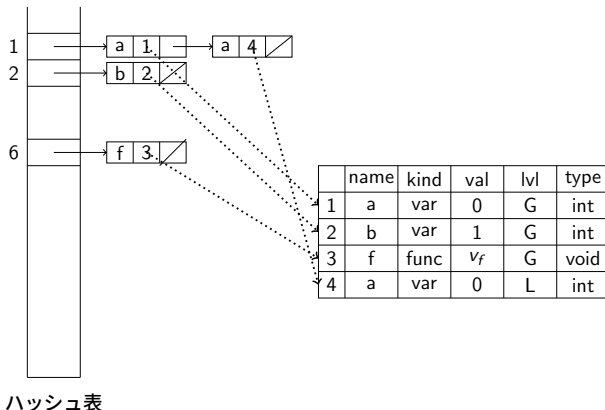
$$h(a_n \cdots a_1) = \left\{ \sum_i^n \text{ord}(a_i) \times 63^{n-1} \right\} \bmod N + 1$$

$N = 1009$ のとき： $h(\text{main}) = h(\text{bqn}) = h(\text{aculvl}) = 949$
リストの実現にはさまざまな方法がある。例 5.4

スコープルールとオープンハッシュ

同じ名前の識別子：同じバケット

オープンハッシュのリストの後ろの方から検索することにする。



型チェック

- ▶ 駆動レコードや記号表において、変数と必要な記憶領域量の対応をとる必要がある。
内部表現：32/64bit 整数，固定小数点，浮動小数点（単精度，倍精度）
- ▶ 演算によって計算するデータの型が異なる。
型によって発行する命令が異なる。データ型が混在するとき，エラーとするか？
- ▶ 構造体，オブジェクトなどのより複雑なデータ型の場合，整合性があるか？リストの内容は同じ型でないといけないか？

変数を通したデータの流れてに応じて型の整合性があるか？

```
int x,z; char y;  
z = x + y;
```

+の両辺の型が違うとき，構文的に正しくても有効な命令に変換可能か？

型定義

T は、整数 (integer) か浮動小数点数 (float) の変数型、および、その配列の型を生成する．例：`int[2][3]`, `float[10]`

生成規則	意味ルール
$T \rightarrow B\ C$	$T.t = B.t, C.b = B.t$
$B \rightarrow \text{int}$	$B.t = \text{integer}$
$B \rightarrow \text{float}$	$B.t = \text{float}$
$C \rightarrow [\text{num}] C_1$	$C.t = \text{array}(\text{number.val}, C_1.t)$ $C_1.b = C_b$
$C \rightarrow \varepsilon$	$C.t = C.b$

`int[2][3]` が割り当てる領域 32bit×6

`float[10]` が割り当てる領域 64bit×10

配列の大きさは予め決定されている必要がある：駆動レコード，ヒープのため

型整合性

型が一致しない場合の対応はプログラミング言語ごとに異なる。

- ▶ 型が一致しない場合はエラーとする。
型ごとに演算を用意する。ML の+(整数) と+. (固定小数点)
OCaml, Haskell, C, Java など
- ▶ 型変換する。
型キャストによって明示的に変換するか、暗黙的に変換する。
部分型の定義によって柔軟に適用させるようにする。
Lisp, SmallTalk, Perl など

型チェック：

- ▶ 静的型チェック コンパイル時に型整合性を検査して、型が合わない場合はコード生成しない。
- ▶ 動的型チェック コンパイル時は整合性を厳しく検査せず、実行時にできるだけ型が合うように実行する。

多相型とオーバーロード

- ▶ 多相型 (polymorphism) : リストなど共通の操作を持つデータ型

List はその要素の型にかかわらず同じ性質を持つ。

結合 (append), 先頭要素 (head), 先頭を除いたリスト (tail),
i 番目の取り出し (i-th) etc.

型変数を持つデータ型: `List<X>`

型変数に具体的なデータ型を適用して, データ型を構成する。

整数リスト = `List<Int>`, 文字リスト = `List<Char>`

- ▶ オーバーロード : 異なる型に対して同じ操作を認める

加算 : 整数, 実数などに対して同じ記号+を用いる

$x+y$ は, x, y の型に応じて操作を切り替える。

x, y がともに整数の場合 : 整数の加算

x, y がともに実数の場合 : 実数の加算

言語によってはオーバーロードなし : ML における `+` と `+`.